

Memories

Learning Goal: External Memory and Peripherals in Verilog.

Requirements: Verilator, GTKWave, CS200 extension

1 Introduction

In this lab, you will implement essential components that will interface with the CPU you developed in the previous lab. Using Verilog, you'll create a synchronous external memory and develop controllers for various peripherals, completing the computer system architecture.

You will implement the following main components:

- A **Memory**: This will be a *large* and relatively *slower* synchronous memory external to the processor. It will serve as both RAM and ROM for your CPU, storing both instructions and data.
- Peripheral Controllers:
 - A **7-Segment LCD Controller**: For managing the 7-segment LCD displays on the Gecko5 board.
 - A **Button Controller**: To interface with the push buttons and switches.
 - An **LED Controller**: For controlling the LED array on the Gecko5 board.

These components will connect to a common system bus, interfacing with the CPU you've already implemented. Figure 1 illustrates this system architecture, showing how your new components will integrate with the existing CPU.

2 Synchronous Memories

In this section, you will implement a *synchronous* RAM module. This RAM will later be incorporated into a larger system that includes other components such as peripheral controllers. FPGAs typically contain several *synchronous* memory blocks (SRAM), which we will emulate to create our RAM module.

The schematic in Figure 1 illustrates the complete system architecture, showing how the RAM you'll implement in this section will eventually integrate with other components, including the CPU and various controllers.

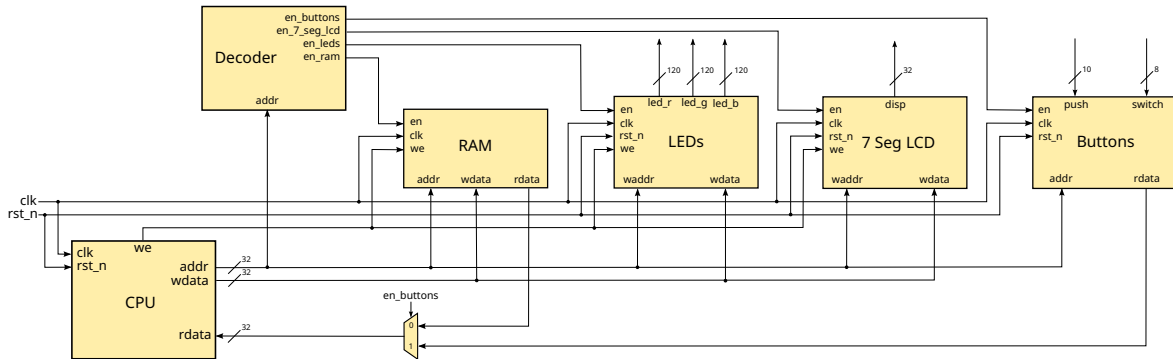


Figure 1: A schema of a system with synchronous memory blocks, LEDs, 7-segment LCD displays, and buttons controllers connected to the CPU.

The **rdata**, **wdata**, and **we** (write enable) signals of this system are shared by all the slave modules (i.e., the memory and the LEDs).

The read process occurs continuously, while the write process is initiated with the **we** signal.

The selection of the module is done by the **Decoder**, which looks at the global address and activates the **en** signal (*enable*) of the corresponding slave module.

The SRAM blocks that we will use on the FPGA have specific read and write timings that are described in detail in the following sections.

2.1 Read Process

The read process of an SRAM is *synchronous* and has a latency of one cycle. The timing diagram in Figure 2 illustrates a typical read process on the bus. During cycle 0, a valid address is provided and the **en** signal is set to 1. The **Decoder** raises the **en** signal, which allows the selected module to send the read data during the next cycle. At the rising edge of the clock, the selected module must save the address and the **en** signal. During cycle 1, the registered address selects the corresponding line in the memory to be outputted on **rdata**. The saved value of **en** enables the output. Another read process may start during cycle 1.

In a *Synchronous* read-memory, when a read address is provided at cycle n , the data will be *valid* for reading at cycle $n + m$, where $m \geq 1$ is the latency of read. Contrarily, in an asynchronous read process, when an address is provided, data is *immediately* (i.e., in the same cycle) valid. Asynchronous memories are only used when the memory is very small (e.g., a register file), because the address decoding is cheap enough (in terms of critical path delay) and can be done within a fraction of a cycle. With larger memories, the address decoding circuit is usually broken down into smaller stages connected by registers. Therefore, more cycles are required to perform the read operation. In fact, the larger the memories are, the bigger m becomes, because more stages of address decoding are required.

2.2 Write Process

The write process of the SRAMs is *synchronous* and has no latency cycles. The diagram in Figure 3 illustrates a typical write process on the bus. It is very similar to a write in the **Register File**. The

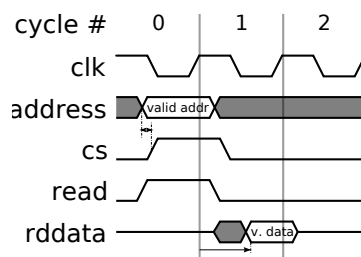


Figure 2: A synchronous read process on the system bus.

address, **wdata** and **we** signals are set during cycle 0. The **Decoder** provides the **en** signal. At the rising edge of the clock, the data is saved by the module. A new writing process can start during cycle 1.

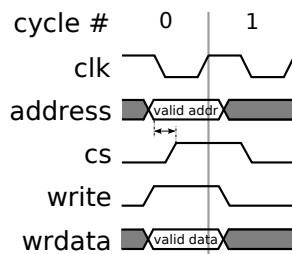


Figure 3: A synchronous write process on the system bus.

2.3 Decoder

The **Decoder** activates one of the slave modules (i.e., the **RAM**, the **Button controller**, the **7-Segment LEDs Controller**, or the **LEDs controller**) at a time, depending on the **address** value and the address space definition as described in Table 1. For example, based on the address space below, the address 0×80000000 selects the RAM. In this case, the **Decoder** will activate the **en_RAM** signal. Remember that at most one **en** signal can be activated at a time.

Table 1: Address space.

0x00000000	
0x4FFFFFFF	
0x50000000	LEDs controller
0x50000FFF	
0x50001000	
0x60000000	7-Segment LEDs controller
0x60000FFF	
0x60001000	
0x70000000	Button controller
0x70000FFF	
0x70001000	
0x80000000	RAM
0x9FFFFFFF	
0xA0000000	
0xFFFFFFF	

2.4 RAM Description

The *synchronous* **RAM** (with read latency of 1 cycle) has a size determined by the parameters in the System Verilog file. Specifically, the RAM spans from address `BASE_ADDR` (32'h80000000) to address `B` (32'hA0000000). This gives the RAM a total size of 0x20000000 bytes, or 512 MiB.

2.4.1 Interface Signals

The RAM module has the following interface signals:

Table 2: RAM Module Interface Signals

Name	Direction	Width	Description
clk	Input	1	Clock
en	Input	1	Enable
we	Input	1	Write enable
addr	Input	32	Address input
wdata	Input	32	Write data input
rdata	Output	32	Read data output

2.4.2 MMIO Region

The RAM occupies an MMIO region starting at base address 0x80000000 and ending at 0x9FFFFFFF as described in Table 1. Each 32-bit word in this region can be read from or written to.

2.4.3 Operation

The RAM operates as follows:

- **Read Process:** The read process is synchronous with a 1-cycle latency. When `en` is asserted and `we` is not asserted in the current cycle, the RAM will prepare to output the data at the given `addr`. This data will be presented on `rdata` in the next clock cycle, regardless of the state of `en` in that next cycle.
- **Write Process:** The write process is synchronous and occurs in the current cycle. When both `en` and `we` are asserted, the RAM will write the data on `wdata` to the address specified by `addr` at the rising edge of the current clock cycle.

The memory is *word-aligned*, meaning data can only be read from or written to it one word (32 bits) at a time. The 2 least significant bits of the address are therefore ignored, as words only begin at addresses that are multiples of four.

It's important to note:

- Once a read operation is initiated (i.e., `en` is asserted and `we` is not asserted in a given cycle), the data will be output in the next cycle even if `en` is deasserted in that next cycle.
- If no read operation was initiated in the previous cycle, `rdata` will output the default value (typically 32'h0).

A SystemVerilog template for the RAM module is provided, which includes some DPI-C functions. These DPI-C functions should be left untouched. You should write your Verilog code for the RAM functionality as if these functions were not present, implementing the RAM behavior using the given interface signals.

2.5 LEDs Controller

The Gecko5 features an LED array comprising 10x12 groups of red/green/blue LEDs. The LEDs controller provides an MMIO interface to control the state of each LED.

2.5.1 Interface Signals

The LEDs controller has the following interface signals:

2.5.2 MMIO Register

The LEDs controller has an MMIO region starting at base address 0x50000000 and ending at 0x50000FFF as described in Table 1. It contains a single 32-bit write-only register to alter LED states.

Table 3: LEDs Controller Interface Signals

Name	Direction	Width	Description
clk	Input	1	Clock
rst_n	Input	1	Reset (active low)
en	Input	1	Enable
we	Input	1	Write enable
waddr	Input	32	Write address
wdata	Input	32	Write data
led_r	Output	120	States for red LEDs
led_g	Output	120	States for green LEDs
led_b	Output	120	States for blue LEDs

2.5.3 Control Register (CTL)

The CTL register, located at offset 0x0 from the base address (i.e., at 0x50000000), controls the state of LEDs. The write data is encoded as follows:

Table 4: CTL Register Encoding for LEDs Controller

Name	Bit field	Description
col	[3:0]	Column selection. 4'b1111 means all columns are selected
row	[7:4]	Row selection. 4'b1111 means all rows are selected
r	[8]	Updating red LEDs
g	[9]	Updating green LEDs
b	[10]	Updating blue LEDs
val	[31:16]	New state of LEDs

The 'val' field is interpreted differently based on the column and row selection:

- If no column or row is selected, all LEDs change their states to val[16].
- If a single column (but not row) is selected, all LEDs in that column change their states to val[25:16].
- If a single row (but not column) is selected, all LEDs in that row change their states to val[27:16].
- If both column and row are selected, the LED changes its state to val[16].

2.5.4 Operation

When the controller receives a write operation to the base address (0x50000000) with the enable (en) and write enable (we) signals asserted, it updates the internal LED states based on the provided write data (wdata).

On reset (when rst_n is low), the controller initializes all LED states to off.

2.5.5 Example Usage

To change the states of all the red LEDs in column 3 to 0001111101 (from row 0 to row 9), one would write the following data to 0x50000000:

- data[3:0] = 4'b0011 (column 3)

- data[7:4] = 4'b1111 (all rows)
- data[8] = 1'b1 (update red)
- data[9] = 1'b0 (don't update green)
- data[10] = 1'b0 (don't update blue)
- data[25:16] = 10'b1011111000 (reverse of 0001111101)

2.6 7-Segment LCD Controller

The Gecko5 features four 7-segment LCD displays. The 7-Segment LCD controller provides a simple MMIO interface for software to control the content of each display.

2.6.1 Interface Signals

The 7-Segment LCD controller has the following interface signals:

Table 5: 7-Segment LCD Controller Interface Signals

Name	Direction	Width	Description
clk	Input	1	Clock
rst_n	Input	1	Reset (active low)
en	Input	1	Enable
we	Input	1	Write enable
waddr	Input	32	Write address
wdata	Input	32	Write data
disp	Output	32	Content of the four 7-segment LCDs

2.6.2 MMIO Register

The LCD controller has an MMIO region starting at base address 0x60000000 and ending at 0x60000FFF as described in Table 1. It contains a single 32-bit write-only register to control the content of LCD displays.

2.6.3 Control Register (CTL)

The CTL register, located at offset 0x0 from the base address (i.e., at 0x60000000), controls the content of the LCDs. The write data is encoded as follows:

Table 6: CTL Register Encoding for 7-Segment LCD Controller

Name	Bit field	Description
lcd_0	[7:0]	Content of LCD display 0
lcd_1	[15:8]	Content of LCD display 1
lcd_2	[23:16]	Content of LCD display 2
lcd_3	[31:24]	Content of LCD display 3

For each LCD display, the 8-bit data controls its content. The encoding of these 8 bits determines which segments of the 7-segment display are lit. Figure 4 shows the mapping of each bit to a segment of the display.

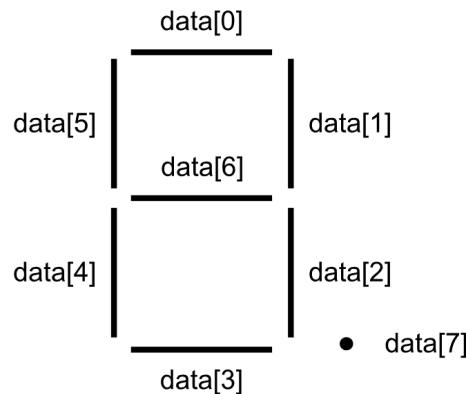


Figure 4: Mapping of data bits to 7-segment display segments

2.6.4 Operation

When the controller receives a write operation to the base address (0x60000000) with the enable (en) and write enable (we) signals asserted, it updates the internal `lcd_content` register with the provided write data (`wdata`). This register directly drives the `disp` output, which controls the four 7-segment LCD displays.

The reset signal (`rst_n`) is synchronous, meaning it is sampled on the rising edge of the clock. When `rst_n` is low, the controller clears all LCD displays (represented by 32'h00000000) on the next rising clock edge.

The controller updates all four displays simultaneously with a single write operation, allowing for efficient updates of the entire display set.

2.6.5 Example Usage

To display the digit "3" on the first LCD display, you would set bits 0, 1, 2, 3, and 6 to 1 for `lcd_0`, resulting in the binary value 01001111 (0x4F in hexadecimal). To set this value, you would write 0x0000004F to the base address 0x60000000.

2.7 Button Controller

The Gecko5 features ten push buttons and eight switches. The Button Controller provides an MMIO interface for software to retrieve the current states of these buttons and switches.

2.7.1 Interface Signals

The Button Controller has the following interface signals:

2.7.2 MMIO Register

The Button Controller has an MMIO region starting at base address 0x70000000 and ending at 0x70000FFF as described in Table 1. It contains a single 32-bit read-only register to retrieve button and switch states.

Table 7: Button Controller Interface Signals

Name	Direction	Width	Description
clk	Input	1	Clock
rst_n	Input	1	Reset (active low)
en	Input	1	Enable
addr	Input	32	Read address
rdata	Output	32	Read data
push	Input	10	States of push buttons
switch	Input	8	States of switches

2.7.3 Value Register (VAL)

The VAL register, located at offset 0x0 from the base address (i.e., at 0x70000000), contains the current states of push buttons and switches. The read data is encoded as follows:

Table 8: VAL Register Encoding for Button Controller

Name	Bit field	Description
push	[9:0]	States of push buttons
switch	[23:16]	States of switches

2.7.4 Operation

The Button Controller operates with a one-cycle latency for read operations, similar to the RAM. The process works as follows:

1. When the controller receives a read operation from the base address (0x70000000) with the enable (en) signal asserted, it captures the current states of the push buttons and switches.
2. In the next clock cycle, it returns these captured states through the rdata output. This one-cycle delay ensures synchronous operation and allows time for the controller to properly sample and prepare the data.

The push buttons are merged into a single 10-bit signal in the following order:

```
{button_top, button_bottom, button_left, button_right, button_center,  
joystick_up, joystick_down, joystick_left, joystick_right, joystick_pressed}
```

The reset signal (rst_n) is synchronous, meaning it is sampled on the rising edge of the clock. While not critical for the current functionality, it can be used to reset the internal val_r register to 0. It's not very useful at the moment but will be more important when you implement more complex functionality in the future.

2.7.5 Example Usage

To read the current states of all buttons and switches, one would perform a read operation from address 0x70000000. The returned 32-bit value in the next clock cycle would contain the states of the 10 push buttons in bits [9:0] and the states of the 8 switches in bits [23:16].

2.8 Exercise

- The system structure is already defined in the project's Verilog files.
- Implement the **Decoder** in the file named `decoder.v`.
- Implement the **Memory** in the file named `mem.v` using a Verilog syntax and **without touching the DPI-C function as mention in the template**.
- Implement the **7-Segment LCD Controller** in the file named `seven_seg_lcd.v`.
- Implement the **Button Controller** in the file named `buttons.v`.
- When every component is implemented, we recommend you to create testbenches for each module and test them separately before passing to the next step.

3 Loading programs and simulating the system

Before proceeding with the simulation, ensure that you have included all the necessary files from your previous labs (ALU and CPU labs). This includes:

- From the ALU lab:
 - `alu.v`
 - `add_sub.v`
 - `comparator.v`
 - `logic_unit.v`
 - `shift_unit.v`
 - `mux4x32.v`
- From the CPU lab:
 - `cpu.v`
 - `controller.v`
 - `register_file.v`
 - `ir.v`
 - `pc.v`
 - `mux2x32.v`

Ensure that all these files are present in your project directory along the files from the template provided for this lab.

To test the complete system, you will use a short RISC-V assembly program and simulate it using the custom debug extension.

- Locate the `program.s` file in your project directory. This file contains a basic RISC-V assembly program.
- Edit `program.s` to test the instructions you've implemented. For example, you might write a program that stores some value into the LEDs memory-mapped address and then check if the output of the LEDs array is correct on the extension. The assembler supports pseudo-instructions such as **li** (load immediate), which can simplify your code. However, be careful when using pseudo-instructions to ensure that the underlying real instructions they expand to are implemented in your CPU. For instance, **li** might expand to **lui** followed by **addi**, so both of these instructions should be supported by your CPU implementation.

```
.section ".text.init"
.globl _start
_start:
    # Load address for LED control (0x50000000)
    li t0, 0x50000000
    # Load the data word (0x00BE23F3)
    li t1, 0x00BE23F3
    # Store the value to the LED control address
    sw t1, 0(t0)
    # Stop the processor
    ebreak
```

- Open the `gecko.v` file in your development environment. This is the top-level module for your system simulation.
- To start debugging:
 - Ensure that `gecko.v` is the active file in your editor.
 - Go to the Debug view in your development environment.
 - Click the “Run” button or press F5 to start the simulation.
- Use the following controls to navigate the simulation:
 - Press F5 again to pause the simulation at any point.
 - Press Shift+F5 to stop the simulation completely.
- As you step through the simulation, observe the behavior of your system. Verify that it correctly executes the instructions in your test program and interacts properly with the peripheral controllers.
- Modify the `program.s` file to test different instructions and scenarios, repeating the simulation process as needed. Try to test all the peripheral interactions you’ve implemented, such as LED control, button input, and 7-segment display output.

For the given example program, you should observe the expected output in Figure 5 if the program is correctly executed by your system. If not, we recommend writing testbenches or simply review the dumped waveforms file at the end of the simulation to see where the problem might be.

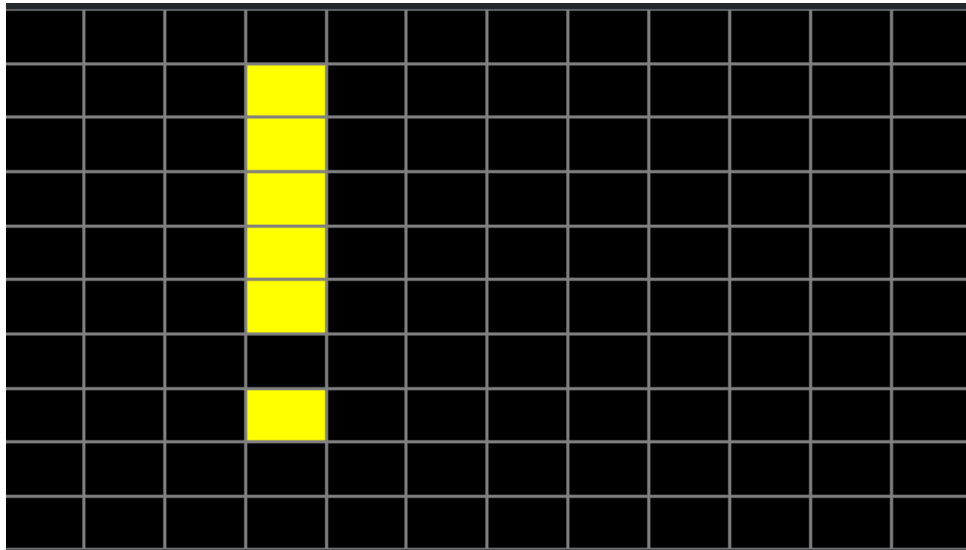


Figure 5: Expected output of the example program.

4 Submission

Submit all Verilog and SystemVerilog files required by the autograder:

- From this lab: `decoder.v`, `mem.sv`, `seven_seg_lcd.v`, `buttons.v`
- From the ALU lab: `add_sub.v`, `comparator.v`, `logic_unit.v`, `shift_unit.v`, `mux4x32.v`
- From the CPU lab: `controller.v`, `register_file.v`, `ir.v`, `pc.v`, `mux2x32.v`

Ensure that all files are included in your submission, as they are necessary for the complete system evaluation.

Once you submit the files, you will receive a report describing the tests that were applied to your design and the results of those tests (success or failure). This submission is a final one, so you will not receive any other feedback. At the end of the results you will get a score that will be used to grade your submission.